

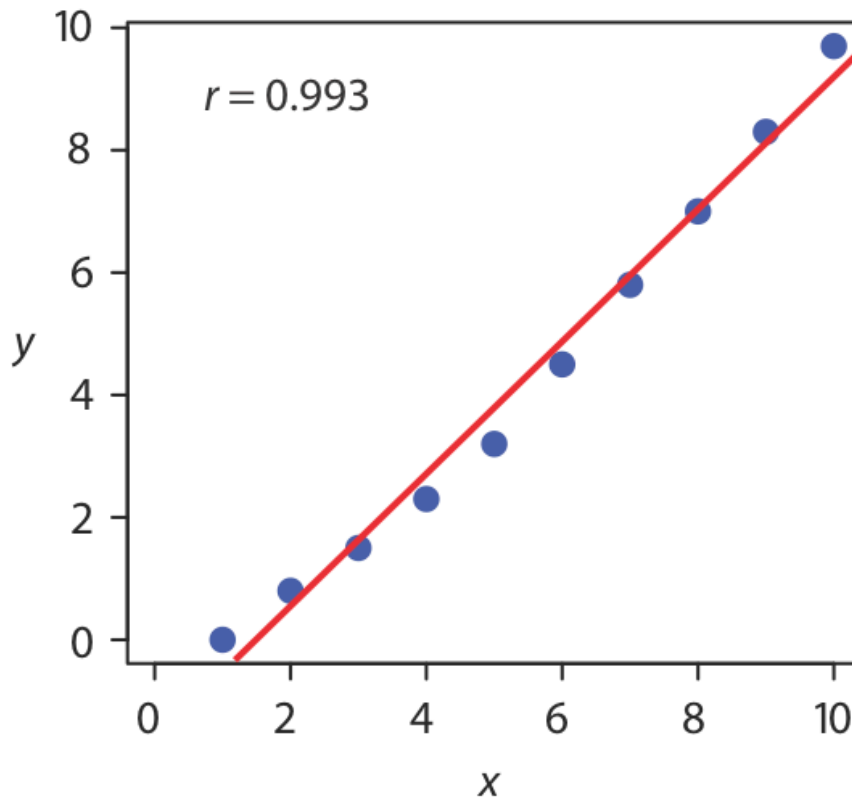
Конспект лекції

▼ Лінійні моделі в задачах регресії

Задача регресії - це задача прогнозування таргету як числової величини. Теоретично прогнозоване значення може бути в діапазоні від мінус нескінченності до плюс нескінченності, але на практиці є певні розумні межі зміни таргету.

Лінійна регресія - це така модель, яка описує набір даних за допомогою лінії, площини, або гіпер-площини (площини, в системі координат з більше, ніж двома незалежними змінними).

Розглянемо набір даних, що складається з двох параметрів: кількості відпрацьованих годин і обсягу виконаної роботи. Проста лінійна регресія має на меті вгадати обсяг виконаної роботи, якщо вказані робочі години. Обирається лінія регресії, яка генерує мінімальну помилку. Також формується лінійне рівняння, яке потім можна використовувати майже для будь-якого набору даних.



▼ **Принципи, які описують мету простої лінійної регресії:**

Проста лінійна регресія використовується для прогнозування зв'язку між змінними в наборі даних і отримання значущих висновків. Проста лінійна регресія в основному використовується для отримання статистичного зв'язку між змінними, який є недостатньо точним. Чотири основні принципи описують використання простої лінійної регресії. Ці принципи перераховані нижче:

1. **Зв'язок між двома змінними вважається лінійним і**

адитивним: Функція прямої лінії встановлюється для кожної пари залежних і незалежних змінних. Нахил цієї лінії відрізняється від значень змінних, доступних у наборі даних. Залежні змінні мають адитивний вплив на значення незалежних змінних.

2. **Помилки є статистично незалежними:** Цей принцип можна розглянути для набору даних, який містить інформацію, пов'язану з часом і рядами. Послідовні помилки такого набору даних не корелюють і є статистично незалежними.

3. Помилки мають постійну дисперсію

(гомоскедастичність): Гомоскедастичність помилок можна розглядати на основі різних параметрів. Ці параметри включають час, інші прогнози та інші змінні.

4. **Нормальність розподілу помилок:** Це важливий принцип, оскільки він підтримує інші три згадані вище. Якщо неможливо встановити зв'язок між змінними в наборі даних або якщо не встановлено будь-який із наведених вище принципів, тоді всі передбачення та висновки, зроблені моделлю, є неправильними. Ці висновки не можуть бути використані надалі в проекті, оскільки реальні результати не будуть отримані, якщо використовуються неправильні та оманливі дані.

▼ Переваги простої лінійної регресії

- Метод надзвичайно простий у використанні, і результати можна отримати без зусиль
- Метод менш складний, ніж інші алгоритми науки про дані, насамперед, якщо відомий зв'язок між залежною та незалежною змінними
- Надмірне перенавчання є поширеним станом, який виникає, коли цей метод приймає безглузду інформацію. Щоб вирішити цю проблему, доступна техніка регуляризації, яка зменшує проблему перенавчання за рахунок зменшення складності

▼ Недоліки простої лінійної регресії

- Хоча проблему перенавчання можна усунути, її не можна ігнорувати. Метод може враховувати безглузді дані, а також втратити значущу інформацію. У такому випадку всі прогнози є висновками про певний набір даних, який буде неправильним, і ефективні результати не можуть бути згенеровані
- Проблема викидів даних також дуже поширена. Викиди вважаються неправильними значеннями, які не відповідають точним даним. Якщо такі значення взяти до уваги, вся модель дасть оманливі результати, які не будуть корисними

- У простій лінійній регресії набір даних вважається незалежним. Це припущення є хибним, оскільки між змінними може існувати певна залежність.

Розглянемо що таке сімейство алгоритмів. Як відомо зі шкільного курсу алгебри, будь-яка пряма може бути описана рівнянням типу $y = b_0 + b_1 * x$. Коефіцієнт b_0 показує в якій точці пряма перетинає вісь ординат, а коефіцієнт b_1 задає тангенс кута нахилу прямої. Запис залежності в загальному вигляді як раз і задає сімейство ліній, або сімейство алгоритмів (моделей). Якщо замість коефіцієнтів ми підставимо конкретні числа - ми опишемо конкретну пряму. Побудова моделі або тренування моделі - це процес визначення (за формулою або ітеративно) значень коефіцієнтів моделі.

Введемо означення, які нам будуть потрібні.

$x = (x^1, x^2, \dots, x^n)$ - набір значень в колонках

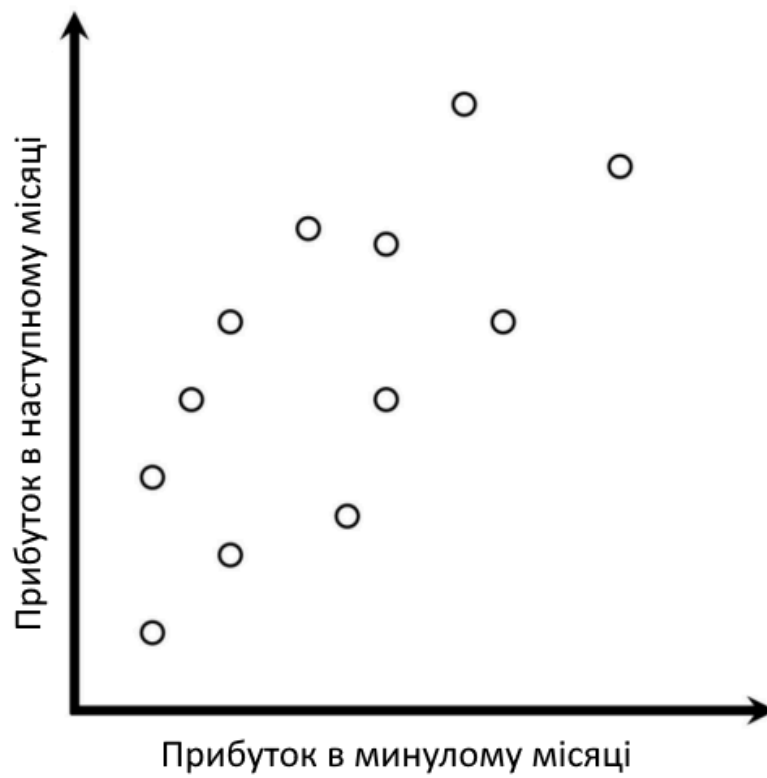
$X = (x_i, y_i)_{i=1}^l$ - тренувальна вибірка даних

$a(x)$ - алгоритм, модель

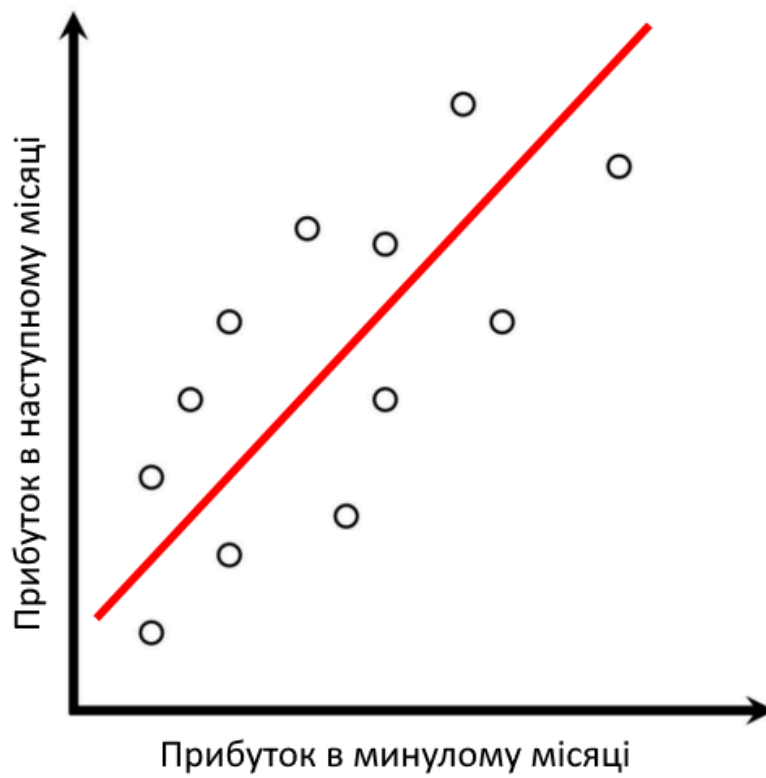
$Q(a, X)$ - функціонал помилки алгоритму a на виборці X (функціонал - це функція вищого порядку, що приймає як аргумент функцію або повертає функцію)

навчання - $a(x) = \operatorname{argmin} Q(a, X)$ при $a \in A$ (такий алгоритм із сімейства алгоритмів, при якому функціонал помилки буде мінімальним)

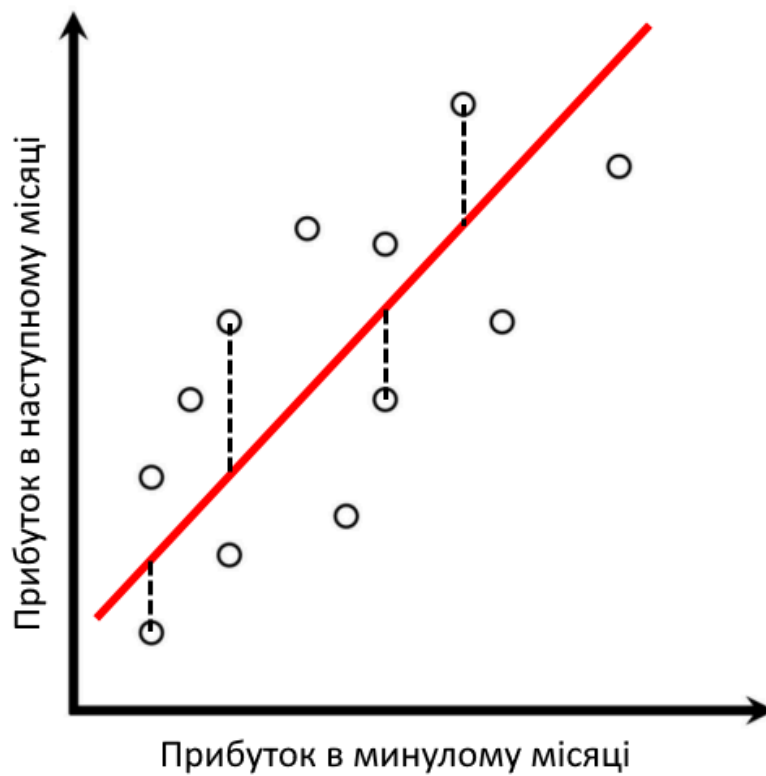
Розглянемо залежність прибутку магазину в наступному місяці відносно прибутку в минулому місяці. Кожна точка на графіку - це пара значень прибутку



Ми не бачимо чіткої лінійної залежності, але все ж таки для таких точок можна провести пряму, що з певною похибкою опише цю залежність

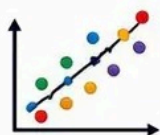


І, хоча червона лінія не проходить через жодну точку, вона досить непогано описує загальну картину залежності очікуваного прибутку від прибутку минулого місяця. Слід зауважити, що похибка прогнозу кожної точки визначається по вертикалі



Лінійна регресія залежної величини від однієї незалежної величини - найпростіший випадок. На практиці частіше зустрічаються множинна регресія, поліноміальна регресія, та робастна регресія.

У випадку множинної регресії в формулу моделі входять декіль незалежних змінних із своїми коефіцієнтами.



MULTIPLE REGRESSION ANALYSIS

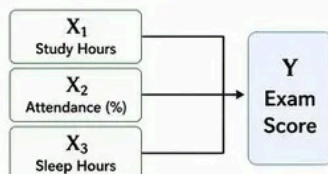
MADE EASY!



Multiple Regression is used to predict a dependent variable (Y) using two or more independent variables ($X_1, X_2, \dots, X_k$). It shows the relationship between Y and each X while holding the other variables constant.

1 WHAT IS MULTIPLE REGRESSION?

It models the linear relationship between one dependent variable and two or more independent variables.



Helps us understand how each factor contributes to the outcome.

2 MODEL / REGRESSION EQUATION

The general form of Multiple Regression Equation:

$$Y = a + b_1X_1 + b_2X_2 + b_3X_3 + \epsilon$$

Where:

Y = Dependent variable (Outcome)
a = Intercept (Constant)
 b_1, b_2, b_3 = Regression coefficients
 X_1, X_2, X_3 = Independent variables
 ϵ = Error term



Goal: Estimate the coefficients (b_1, b_2, b_3) to predict Y as accurately as possible.

3 EXAMPLE STUDY

We want to predict Exam Score (Y) from:

- ✓ Study Hours per day (X_1)
- ✓ Attendance (%) (X_2)
- ✓ Sleep Hours per day (X_3)



Sample Data (n = 10 Students)

Student	Study Hours (X_1)	Attendance (%) (X_2)	Sleep Hours (X_3)	Exam Score (Y)
1	2	70	6	55
2	3	75	7	62
3	4	80	6	68
4	5	85	7	74
5	6	90	8	82
6	2	65	5	50
7	3	70	6	58
8	4	75	7	64
9	5	80	7	72
10	6	90	8	84

4 ASSUMPTIONS

- Linearity**
Relationship between Y and each X is linear.
- Independence**
Observations are independent of each other.
- Normality**
Residuals are normally distributed.
- Homoscedasticity**
Constant variance of residuals at all levels of X.
- No Multicollinearity**
Independent variables are not highly correlated with each other.



If assumptions are violated, results may be unreliable.

5 REGRESSION OUTPUT (Example)

Model Summary

R	R Square	Adjusted R Square	Std. Error of the Estimate
0.948	0.899	0.874	3.21

Interpretation:

- $R^2 = 0.899 \rightarrow 89.9\%$ of the variation in Exam Score is explained by Study Hours, Attendance and Sleep Hours.
- Adjusted $R^2 = 0.874$ (adjusts for number of predictors).
- Std. Error = 3.21 (average prediction error).

Coefficients^a

Predictor	Unstandardized Coefficients		Standardized Coefficients Beta	t	Sig. (p-value)
	B	Std. Error			
(Constant)	-15.245	5.432	-	-2.81	0.022
Study Hours (X_1)	6.250	0.842	0.542	7.42	0.000
Attendance (X_2)	0.412	0.071	0.381	5.80	0.001
Sleep Hours (X_3)	1.983	0.658	0.204	3.02	0.016

a. Dependent Variable: Exam Score (Y)

6 INTERPRETATION OF COEFFICIENTS

- Intercept ($a = -15.245$)**
If Study Hours, Attendance and Sleep Hours are all 0, the predicted Exam Score is -15.245 (not meaningful in real life, but required for the equation).
- Study Hours ($b_1 = 6.250$)**
Holding Attendance and Sleep Hours constant, for each additional 1 hour of study, Exam Score increases by 6.250 points on average. ($p = 0.000 < 0.05 \rightarrow$ Significant)
- Attendance ($b_2 = 0.412$)**
Holding other variables constant, for each 1% increase in Attendance, Exam Score increases by 0.412 points on average. ($p = 0.001 < 0.05 \rightarrow$ Significant)
- Sleep Hours ($b_3 = 1.983$)**
Holding other variables constant, for each additional 1 hour of sleep, Exam Score increases by 1.983 points on average. ($p = 0.016 < 0.05 \rightarrow$ Significant)

✓ All p-values $< 0.05 \rightarrow$ All predictors are statistically significant.

7 REGRESSION EQUATION (From Output)

$$Y = a + b_1X_1 + b_2X_2 + b_3X_3$$

$$\text{Exam Score} = -15.245 + 6.250(\text{Study Hours}) + 0.412(\text{Attendance \%}) + 1.983(\text{Sleep Hours})$$

8 PREDICTION EXAMPLE

Predict Exam Score for a student who has:
Study Hours = 5, Attendance = 85%, Sleep = 7

$$Y = -15.245 + 6.250(5) + 0.412(85) + 1.983(7)$$

$$Y = -15.245 + 31.250 + 35.020 + 13.881$$

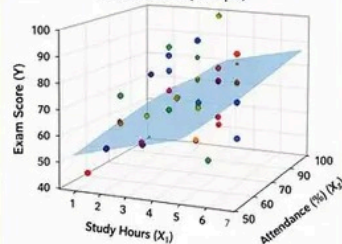
$$Y = 64.906 \approx 64.91$$

Predicted Exam Score = 64.91



9 VISUAL: RELATIONSHIP

3D Scatter Plot (Example)



(Holding Sleep Hours constant for visibility)



Higher study hours and attendance are associated with higher exam scores.

10 GOODNESS OF FIT

R^2 (Coefficient of Determination)

$R^2 = 0.899 \rightarrow 89.9\%$ of the variation in Exam Score is explained by the three predictors. 10.1% is due to other factors (error).

Adjusted R^2

Adjusted $R^2 = 0.874 \rightarrow$ More suitable when comparing models with different numbers of predictors.

Standard Error of Estimate

SEE = 3.21 $\rightarrow$ On average, predicted scores differ from actual scores by about 3.21 points.



11 KEY TAKEAWAYS



- ✓ Multiple Regression predicts Y using multiple X variables.
- ✓ Each coefficient shows the unique effect of a predictor while holding others constant.
- ✓ Use R^2 and Adjusted R^2 to check how well the model fits.
- ✓ Check assumptions before trusting the results.
- ✓ Use the equation to make predictions and support decisions.

WHEN TO USE MULTIPLE REGRESSION?

- ✓ When more than one factor affects the outcome.
- ✓ When you want to know the contribution of each factor.
- ✓ When you want better prediction accuracy.



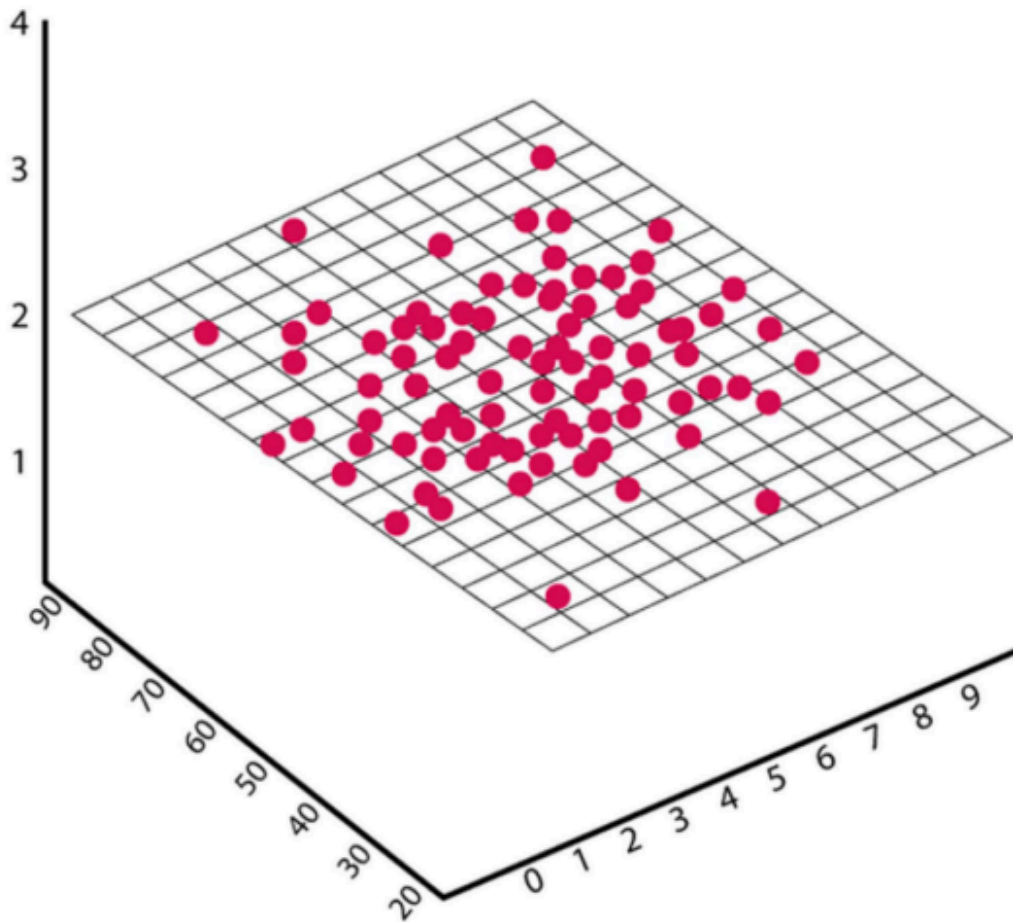
AFTER SIGNIFICANT MODEL?

If the overall model is significant (F test), proceed with individual predictor interpretation and check multicollinearity (VIF < 5 is recommended).

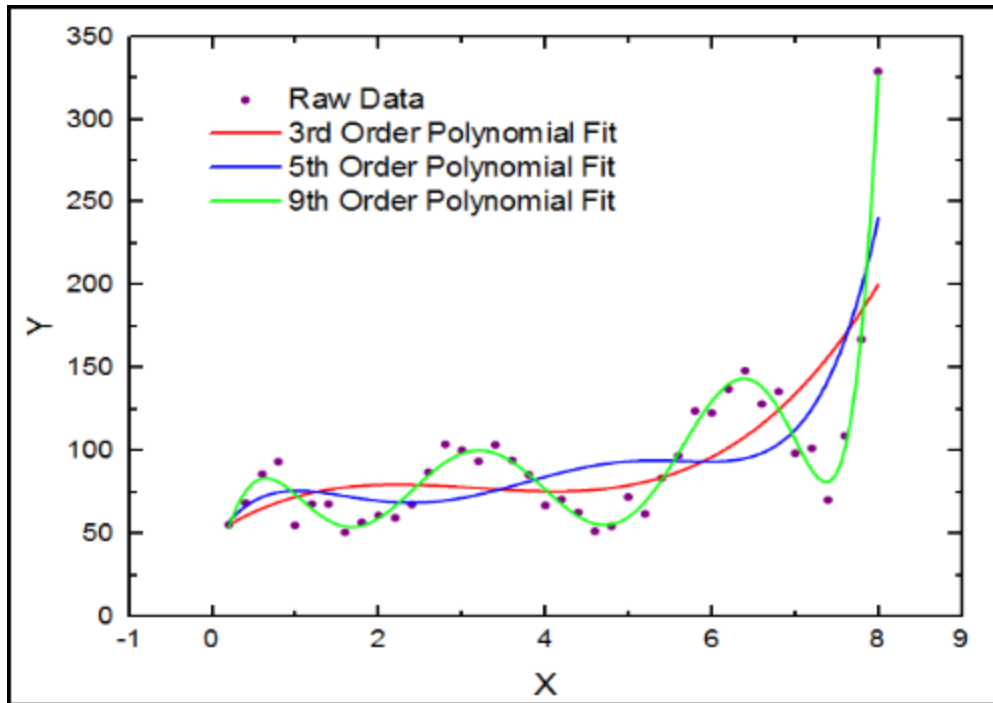
Use diagnostics (residual plots, normal P-P plot, VIF) to validate the model.



Для множинної регресії моделі буде відповідати не пряма лінія, а площина (в загальному випадку - гіперплощина).



У випадку поліноміальної регресії як додаткові незалежні змінні використовуються ступені оригінальної незалежної змінної.

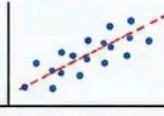
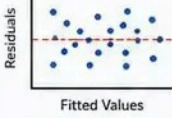
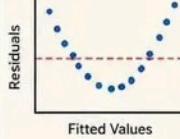
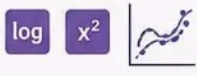
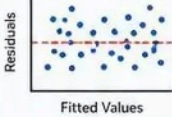
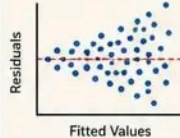
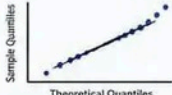
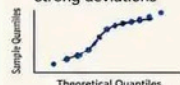
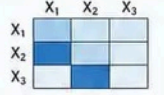
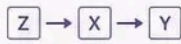


Робастна регресія використовується коли дані мають багато викидів, які погіршують якість прогнозу лінійної моделі.

Не завжди дані можуть бути описані за допомогою лінійної регресії. Після побудови моделі слід перевірити ряд припущень. Якщо якісь із припущень не відповідають дійсності слід спробувати трансформацію незалежних змінних та таргету або тренувати моделі іншого типу.

ASSUMPTIONS OF LINEAR REGRESSION (OLS)

These assumptions help ensure that the estimated coefficients are unbiased and that hypothesis tests, confidence intervals, and predictions are reliable.

#	ASSUMPTION	MEANING	HOW TO CHECK	SIGNS OF VIOLATION	POSSIBLE REMEDIES
1	LINEARITY  The relationship between predictors (X) and the outcome (Y) is linear in the parameters.	$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \varepsilon$ The model is linear in the parameters, not necessarily in the variables themselves.	- Scatterplots of Y vs X - Residuals vs Fitted Values plot 	Curved pattern in residual plot 	- Transform variables (log, square root, etc.) - Add polynomial or interaction terms - Use nonlinear models 
2	INDEPENDENCE OF OBSERVATIONS  Errors (residuals) are independent of one another.	The value of one observation (or residual) is not related to another. Important in time series, repeated measures, and panel data.	- Study design - Durbin-Watson test (for time series) Durbin-Watson statistic ≈ 2 No autocorrelation	Pattern or clustering in residuals over time/order 	- Use models that account for correlation (e.g., AR models, mixed models) - Add relevant variables - Newey-West or other robust SEs 
3	HOMOSCEDASTICITY (CONSTANT VARIANCE)  The variance of residuals is constant across all levels of fitted values.	Formally, $\text{Var}(\varepsilon_i) = \sigma^2$ (a constant) across all i .	- Residuals vs Fitted Values plot - Breusch-Pagan test 	Funnel shape (variance increases or decreases) 	- Transform dependent variable (Y) - Weighted Least Squares - Robust (heteroskedasticity-consistent) standard errors 
4	NORMALITY OF RESIDUALS  Residuals are approximately normally distributed.	Needed mainly for valid confidence intervals and hypothesis tests in small samples. Not required for unbiased coefficients.	- Histogram of residuals - Normal Q-Q plot  	- Histogram not bell-shaped - Q-Q plot shows strong deviations  	- Transform dependent variable (Y) - Use larger sample - Use bootstrap methods for inference 
5	NO PERFECT MULTICOLLINEARITY  Predictors are not exact linear combinations of each other.	No predictor can be written as a perfect linear combination of others.	- Correlation matrix - Variance Inflation Factor (VIF) $VIF < 5$ usually OK $VIF > 10$ serious concern	- Very high correlations between predictors - VIF is very large Consequences: - Unstable coefficients - Large standard errors - Hard to interpret individual effects 	- Remove or combine correlated predictors - Use Principal Component Regression (PCR) - Ridge/Lasso regression 
6	EXOGENEITY (ZERO CONDITIONAL MEAN)  $E(\varepsilon X) = 0$ Errors have no systematic relationship with predictors.	All factors that affect Y are either included in the model or are unrelated to the predictors.	Based on subject-matter knowledge and study design No statistical test can fully verify this assumption.	Violations occur when: - Important variables are omitted - Measurement error in predictors - Reverse causality Consequences: - Biased coefficients - Misleading conclusions	- Include relevant variables - Improve measurement - Use instrumental variables (IV) or natural experiments when appropriate 


KEY TAKEAWAY

Check assumptions using plots, tests, and your knowledge of the data and context. No model is perfect—assess whether any violations are serious enough to affect your conclusions.

WHAT EACH GROUP OF ASSUMPTIONS GIVES YOU


Needed for unbiased estimates:

- Linearity
- Independence
- Exogeneity
- No perfect multicollinearity


Needed mainly for valid inference (p-values, CIs):

- Homoscedasticity
- Normality of residuals

▼ Навчання моделі лінійної регресії

Модель лінійної регресії може навчатись різними методами. Один із них - аналітичний. В загальному випадку рівняння матиме наступний вигляд:

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_K x_K + u,$$

де u - певна випадкова похибка.

Для кожного рядка з тренувальної вибірки даних можемо записати подібне рівняння. Отримаємо систему з n рівнянь. І цю систему ми можемо переписати у вигляді матриць

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad X = \begin{pmatrix} x'_1 \\ x'_2 \\ \vdots \\ x'_n \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1K} \\ 1 & x_{21} & \cdots & x_{2K} \\ \vdots & \ddots & \vdots & \\ 1 & x_{n1} & \cdots & x_{nK} \end{pmatrix}, \quad \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_K \end{pmatrix}, \quad u = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{pmatrix}.$$

В якості функціонала похибки можемо обрати метод найменших квадратів. Шукані коефіцієнти β_i знаходяться з рівняння

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \left| y_i - \beta_0 - \sum_{j=1}^K X_{ij} \beta_j \right|^2 = \arg \min_{\beta} \|y - X\beta\|^2.$$

Аналітичний розв'язок в матричній формі записують у вигляді

$$\hat{\beta} = (X'X)^{-1} X'y.$$

Це лише один із методів визначення вагових коефіцієнтів лінійної регресії. До недоліків можна віднести обчислювальну складність такого підходу, але на сучасних комп'ютерах ак обчислення відбуваються достаньо швидко.

▼ Лінійна регресія в scikit-learn

Пакет scikit-learn для роботи з моделями класичного машинного навчання має відповідний клас, що тренує модель лінійної регресії

Розглянемо невеликий приклад. Візьмемо кілька точок і побудуємо на цих точках значення таргета за заданною лінійною формулою, додавши трохи збурень

```
X = np.array([[1, 0], [1, 2], [2, 0], [2, 2]])
# y = -1 + 2.5 * x_1 - 3 * x_2
y = -1 + np.dot(X, np.array([2.5, -3])) + np.random.normal
(0, 0.4, 4)
y
>>>array([ 1.03170413, -4.37163359,  4.1316081 , -1.414183
26])
```

Тепер створимо екземпляр LinearRegressor і натренуємо його на наших даних

```
reg = LinearRegression().fit(X, y)
reg.score(X, y)
>>>0.9993583592621068
```

Отримане значення - коефіцієнт детермінації або R^2 показує наскільки отримана модель описує значення (максимальне значення 1.0, нормальним вважають значення в діапазоні від 0.25 до 0.4, дуже поганими є від'ємні значення)

Перевіримо які значення вагових коефіцієнтів в натренованій моделі

```
reg.intercept_, reg.coef_
>>>(-1.9613596107023397, array([ 3.02867715, -2.7372822
7]))
```

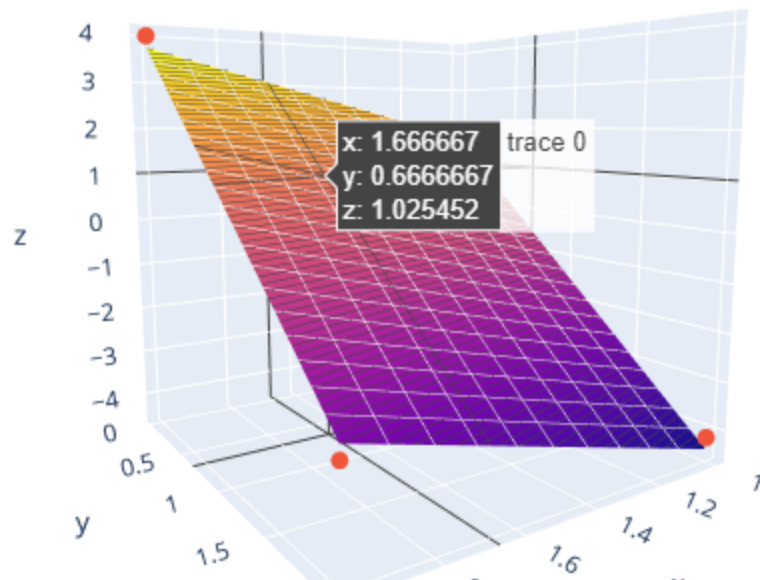
Як бачимо, отримані вагові коефіцієнти відрізняються від тих, які ми прописали в формулі. Задані нами випадкові відхилення спрацювали. Перевіримо наскільки близькі значення спрогнозувала модель

```
y_pred = reg.predict(X)
df = np.hstack([X, y.reshape(4,1), y_pred.reshape(4, 1)])
```

```
df
>>>array([[ 1.          ,  0.          ,  1.03170413,  1.06731
754],
          [ 1.          ,  2.          , -4.37163359, -4.407247
],
          [ 2.          ,  0.          ,  4.1316081 ,  4.0959946
9],
          [ 2.          ,  2.          , -1.41418326, -1.3785698
5]])
```

Так, прогнози відрізняються, але не суттєво.

Побудуємо графік площини, яку описує отримана нами модель, і нанесемо точки, на яких вона тренувалась



▼ Градієнтний спуск в задачах лінійної регресії

Іншим поширеним методом тренування моделей в машинному навчанні є градієнтний спуск (gradient descent). В основі цього методу лежить ітеративний підхід. Тобто, при навчанні на кожній ітерації отримують нові значення вагових коефіцієнтів, розраховують похибку і повторюють ітерації до тих пір, поки похибка моделі не буде менша за задану

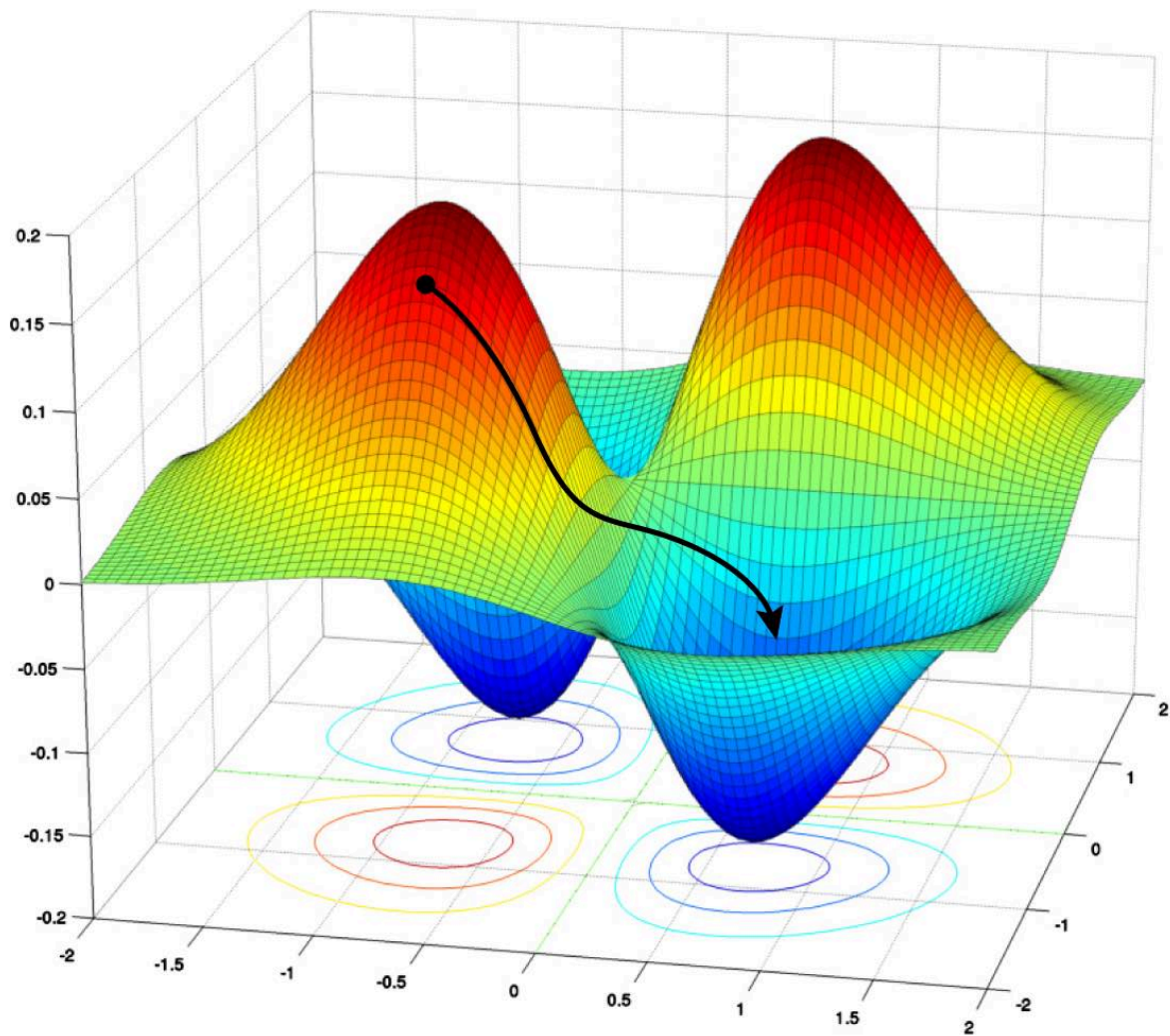
Уявімо, що ми тренуємо модель із двома незалежними змінними. В якості функціоналу похибки найчастіше використовують середньоквадратичну похибку (MSE). Ми не знаємо якою функцією можна описати поверхню, яка складається із множини точок, що отримують при підстановці всіх можливих значень вагових коефіцієнтів моделі. Але ми знаємо, що ця поверхня випукла і гладка. Тобто в кожній точці цієї поверхні ми можемо розрахувати градієнт

Градiєнт — міра зростання або спадання в просторі якоїсь фізичної величини на одиницю довжини. Градієнт, як завжди вважають, — векторна величина, яка визначає в кожній точці простору не лише швидкість зміни, а й напрямок найшвидшої зміни функції, що залежить від координат. Для скалярного поля $U(x, y, z)$ градієнт визначається за формулою

$$\nabla U = \text{grad}U = \frac{\partial U}{\partial x}i + \frac{\partial U}{\partial y}j + \frac{\partial U}{\partial z}k ,$$

де i, j, k - орти системи відліку

Напрямок вектору градієнта показує напрямок, в якому функція найшвидше зростає. А нам для тренування необхідно знати напрямок найшвидшого зменшення значення функції помилки. На графіку нижче схематично зображено поверхню, що описує функціонал похибки, та бажану траєкторію руху по поверхні для знаходження мінімуму цього функціонала. Зауважимо, що ми не знаємо аналітичного запису функціонала похибки і фактично знаємо значення в конкретних точках (підставляючи значення вагових коефіцієнтів) і градієнт в цих точках

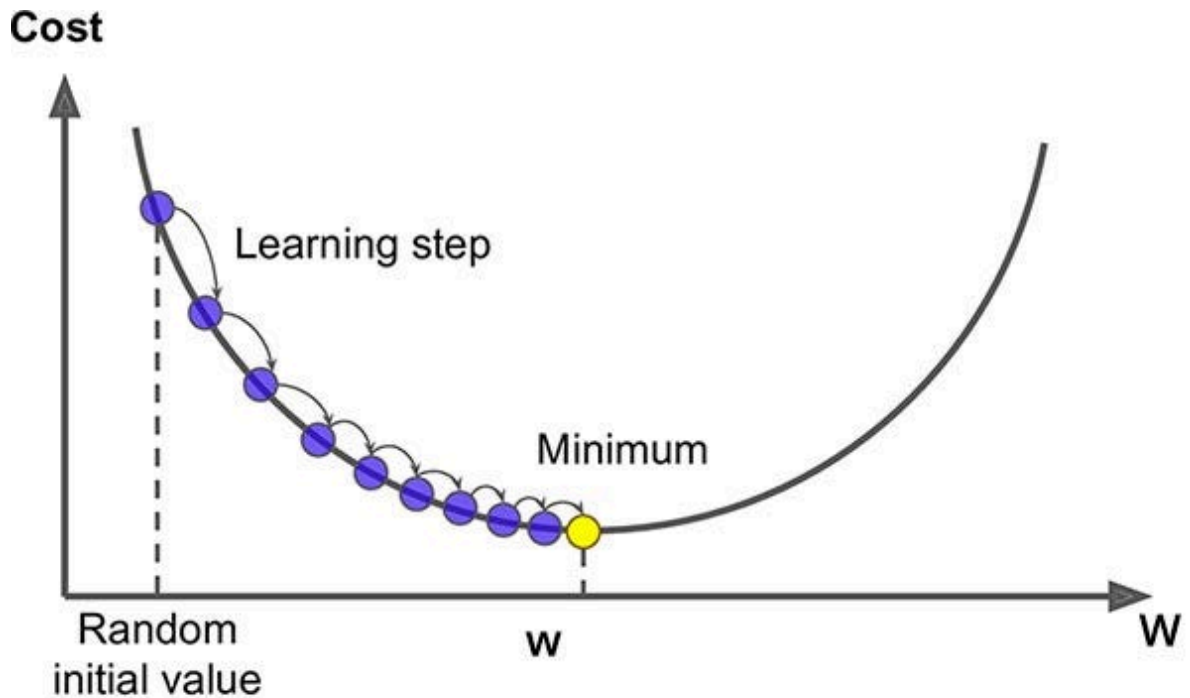


Алгоритм тренування можна записати наступним чином:

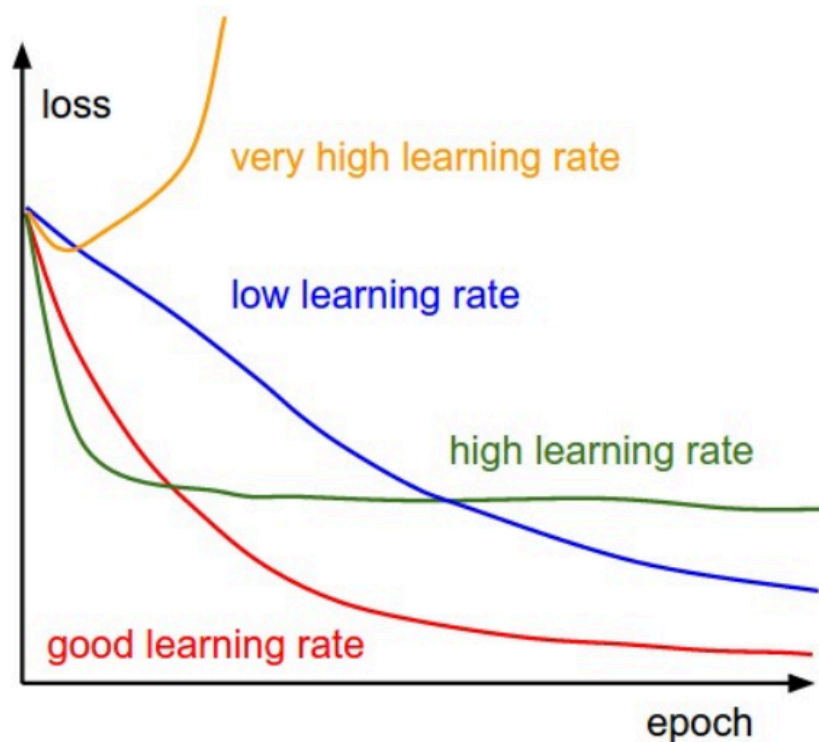
1. Призначаємо ваговим коефіцієнтам значення 0 ($w^0 = 0$)
2. Для кожної ітерації $t = 1, 2, 3, \dots, n$ розраховуємо градієнт функції похибки в точці, координати якої описуються ваговими коефіцієнтами і змінюємо значення вагових коефіцієнтів за формулою
3. $w^t = w^{t-1} - \eta_t \nabla Q(w^{t-1}, X)$, де η_t - крок зміни вагових коефіцієнтів (learning rate)
4. Розраховуємо похибку при отриманих вагових коефіцієнтах і повторюємо ітерації до тих пір, поки значення похибки не буде в

межах допустимого значення (або досягнемо граничного значення ітерацій)

Підбір кроку зміни вагових коефіцієнтів - окрема тема. Тут зазначимо, що занадто великий крок може не привести нас до мінімуму функціонала похибки взагалі, а занадто малий крок - суттєво збільшить кількість ітерацій



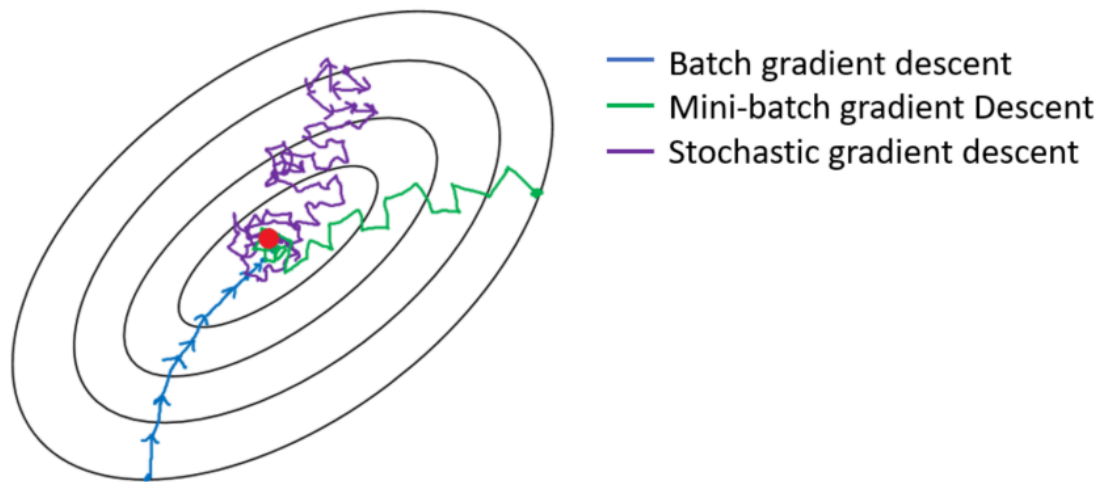
Зазначимо, що при розрахунку градієнту в точці, ми маємо враховувати всі точки в датасеті. Відповідно, із збільшенням датасету, час тренування моделі буде збільшуватись. В даному методі ми на кожній ітерації зменшуємо похибку, тому графік буде мати наступний вигляд (крім крайнього випадку, коли взяли дуже великий крок)



▼ Стохастичний і mini-batch градієнтний спуск

На практиці при тренуванні лінійних моделей частіше використовують алгоритм не повного градієнтного спуску, а спрощені алгоритми - стохастичний і mini-batch градієнтний спуск

В алгоритмі стохастичного градієнтного спуску градієнт функціоналу похибки розраховують не по всьому датасету, а по одному випадковому рядку. При цьому, напрямок градієнту не вказує на точний напрямок збільшення функціоналу похибки, тому і крок ми будемо робити не зовсім в напрямку мінімуму функціоналу, а достатньо приблизно. Кількість ітерацій при використанні цього алгоритму збільшується в рази, але кожна ітерація виконується набагато швидше. Схему кроків зміни вагових коефіцієнтів моделі зображено фіолетовим кольором. Як бачимо кроки можуть відбуватись і в напрямку від точки мінімуму



В алгоритмі mini-batch градієнтного спуску розрахунок градієнту проводять не по одній випадковій точці, а по невеликій підвиборці випадкових точок (зазвичай від 50 до 256 точок). Схема зміни вагових коефіцієнтів показана зеленим кольором. Як бачимо, мінімуму алгоритм досягає за меншу кількість ітерацій ніж для стохастичного градієнтного спуску

```
import numpy as np
from sklearn.linear_model import SGDRegressor
n_samples, n_features = 10, 5
rng = np.random.RandomState(0)
y = rng.randn(n_samples)
X = rng.randn(n_samples, n_features)
reg = SGDRegressor(max_iter=1000, tol=1e-3)
reg.fit(X, y) # SGD algo
#reg.partial_fit(X,y) # mini-batch GD
```

▼ Метрики в лінійній регресії

METRICS FOR REGRESSION

IN MACHINE LEARNING



Regression metrics evaluate how well a model predicts continuous values. They help us understand accuracy, error magnitude, and goodness of fit.

METRIC	FORMULA	WHERE	INTERPRETATION & VISUAL
MAE (Mean Absolute Error) - Average of absolute differences between actual and predicted values. - Easy to understand and interpret.	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	n = number of observations y_i = actual value $\hat{y}_i$ = predicted value Best When: You want to treat all errors equally.	Lower is better. 0 is perfect.
MSE (Mean Squared Error) - Average of squared differences. - Penalizes larger errors more.	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	n = number of observations y_i = actual value $\hat{y}_i$ = predicted value Best When: You want to heavily penalize large errors.	Lower is better. 0 is perfect.
RMSE (Root Mean Squared Error) - Square root of MSE. - Returns error in the same unit as the target variable.	$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	n = number of observations y_i = actual value $\hat{y}_i$ = predicted value Best When: You want an interpretable error metric in original units.	Lower is better. 0 is perfect.
R² (R-squared / Coefficient of Determination) - Measures the proportion of variance in the target explained by the model. - Goodness of fit metric.	$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$	$\bar{y}$ = mean of actual values y_i = actual value $\hat{y}_i$ = predicted value Best When: You want to understand how well the model explains the variance.	Higher is better. 1 → Perfect fit 0 → No better than mean < 0 → Worse than mean
Adjusted R² - Adjusts R² for the number of predictors in the model. - Prevents overfitting by adding irrelevant features.	$\text{Adjusted } R^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$	n = number of observations p = number of predictors R^2 = R-squared Best When: You are comparing models with different numbers of features.	Higher is better. Ranges from -∞ to 1. Closer to 1 is better.



KEY TAKEAWAYS

Use MAE
for interpretability and robustness to outliers.

Use MSE
when large errors should be heavily penalized.

Use RMSE
for error in original units; sensitive to outliers.

Use R²
to measure how well the model explains variance.

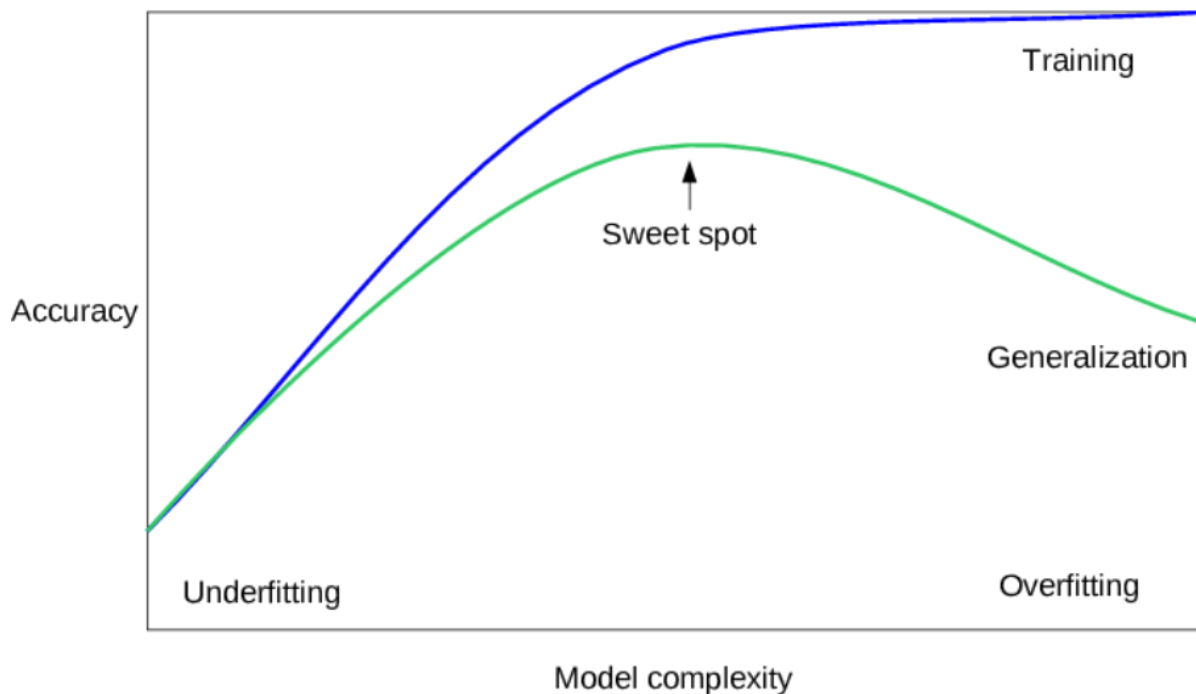
Use Adjusted R²
to compare models fairly and avoid overfitting.

GENERAL TIPS

- ✓ Always visualize predictions vs actuals.
- ✓ Understand your business context before choosing a metric.
- ✓ Use multiple metrics for a complete evaluation.

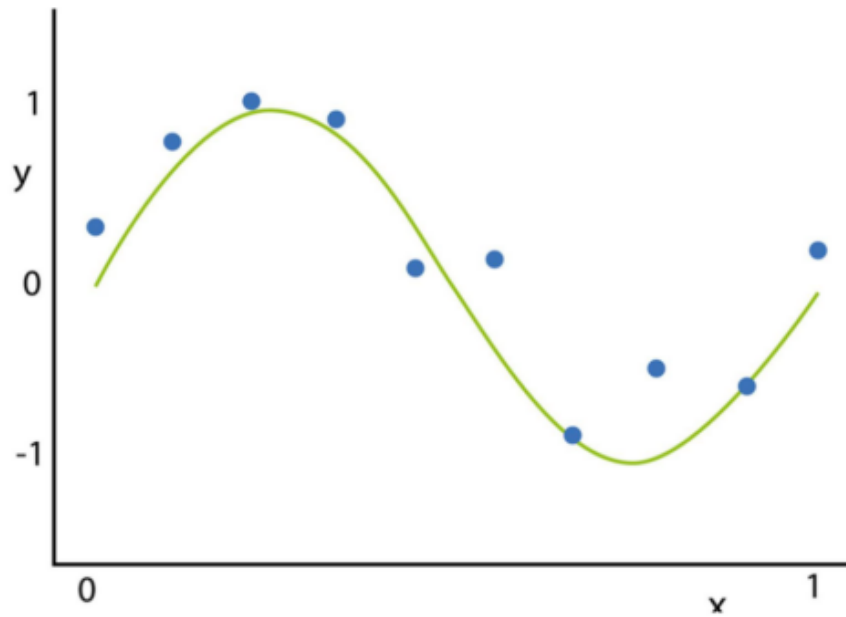
▼ Перенавчання і недонавчання. Гіперпараметри

При тренуванні будь-якої моделі алгоритм намагається якомога краще описати отримані дані. Але проблема полягає в тому, що навчання відбувається на одних даних, а далі модель працює із тими даними, яких вона не бачила при навчанні. Тому виникає ситуація, коли модель показує гарні результати при навчанні і погані на даних, яких вона не бачила. Ця ситуація називається перенавчанням (overfitting), або кажуть "модель погано генералізує дані"

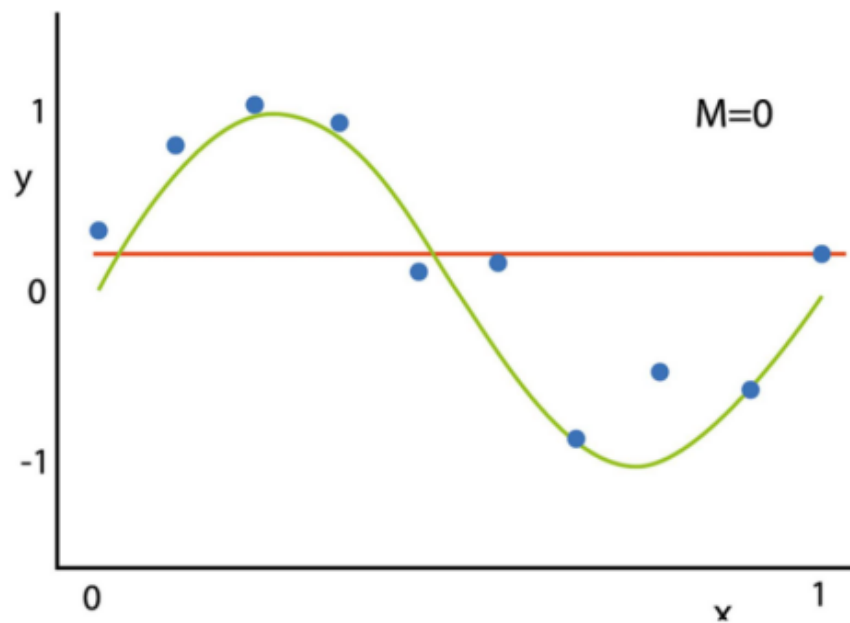


Ця ситуація показана в правій частині графіка. Ми можемо певним чином "спростити" модель, щоб вона не дуже сильно підлаштовувалась під дані, на яких вона тренується. Якщо захопитись спрощенням, то можна перейти в ліву частину графіка. Модель показує погані результати і на даних, на яких вона тренується, і на нових даних. Наша задача полягає в знаходженні такої моделі, яка на нових даних показує максимальну точність (sweet spot на графіку)

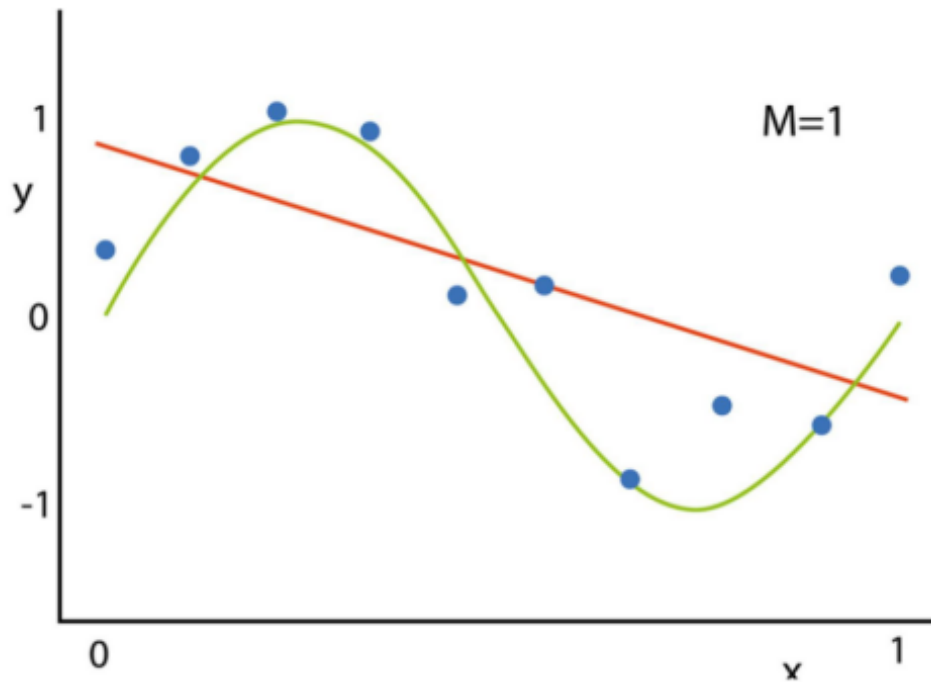
Розглянемо наступний приклад. Візьмемо точки, що приблизно описані синусоїдою



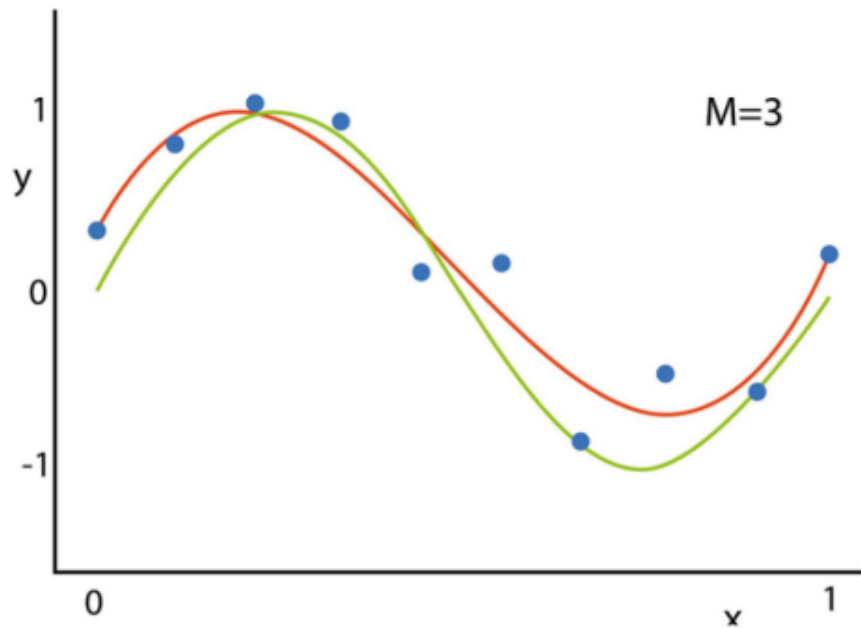
Спробуємо описати ці точки прямою $y = w_0$



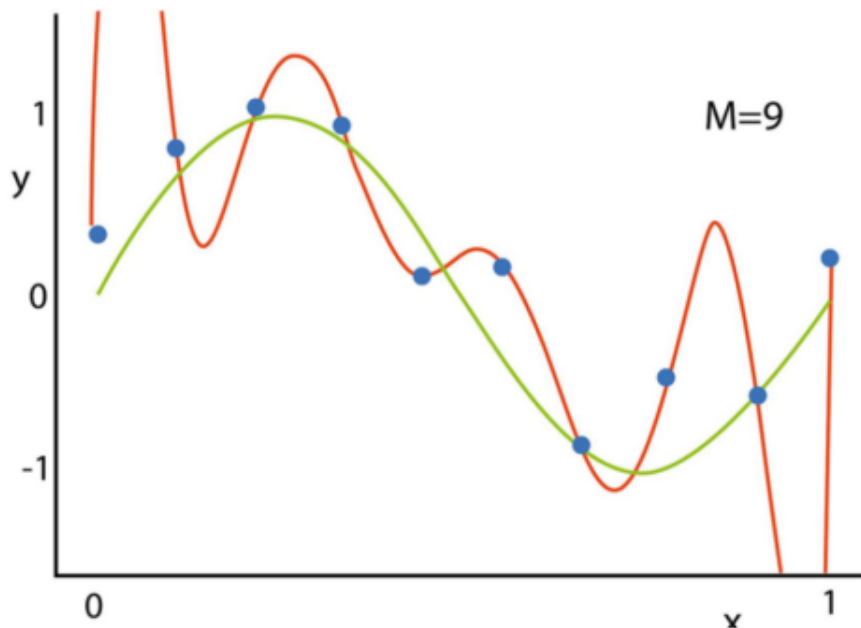
Це недонавчання (underfitting). Ми явно погано опишемо дані. Ускладнимо модель, додамо одну змінну: $y = w_0 + w_1x$



Не дуже допомогло, хоча модель уже не константна. Спробуємо ще ускладнити модель. Але у нас відсуня друга незалежна змінна. Згадаємо, що ми можемо створювати нові змінні на базі існуючих. І аналізуючи графік, можемо сказати, що на даному відрізку точки нагадують графік поліноміальної функції. Запишемо нашу модель у вигляді полінома 3-го ступеня: $y = w_0 + w_1x + w_2x^2 + w_3x^3$



Так значно краще. А якщо продовжувати ускладнювати модель і довести поліном до 9-го ступеня, оскільки ми маємо 10 точок? $y = w_0 + w_1x + w_2x^2 + \dots + w_9x^9$



Бачимо, що графік отриманої функції точно пройшов через всі наші 10 заданих точок, але форма графіка дуже сильно відрізняється від зеленої

кривої. Ми отримали перенавчання і маємо зупинитись на попередній версії моделі

Глянувши на ті, моделі, які ми записали, можемо помітити, що всі вони є лінійними моделями, оскільки складаються із доданків, отриманих множенням вагового коефіцієнта на змінну (оригінальну чи створену нами). І різниця полягає лише в кількості доданків. Що є кількість доданків для моделі? Це називають **гіперпараметром**.

Гіперпараметрами моделі називаються так параметри, які не залежать від даних, на яких модель навчається, а необхідні для керування процесом навчання

На цьому ж прикладі ми також бачимо, що для тренування лінійної моделі ми використали певні нелінійні перетворення. В даному випадку, маючи одну змінну, ми описали модель як поліном. Поліноміальна регресія є окремим випадком лінійної регресії. Аналогічно ми можемо використати для опису нових змінних експоненту, синус та інші функції. В загальному випадку можна сказати, що ми за допомогою нелінійних перетворень "викривили" простір наших незалежних змінних з метою отримання кращих результатів при описі даних лінійною моделлю

▼ Регуляризація

Отже, ми розуміємо, що модель не повинна бути дуже простою і не повинна бути дуже складною. Чи можемо ми якимось чином вплинути на процес тренування? Давайте спочатку візуально глянемо якими поліномами 3-го і 9-го ступеня ми описали дані з попереднього параграфа

$$y = 0.4 + 8x - 23x^2 + 19x^3$$

$$y = 0.5 + 1486521x - 28954724x^2 + 48613258x^3 + \dots + 2740x^9$$

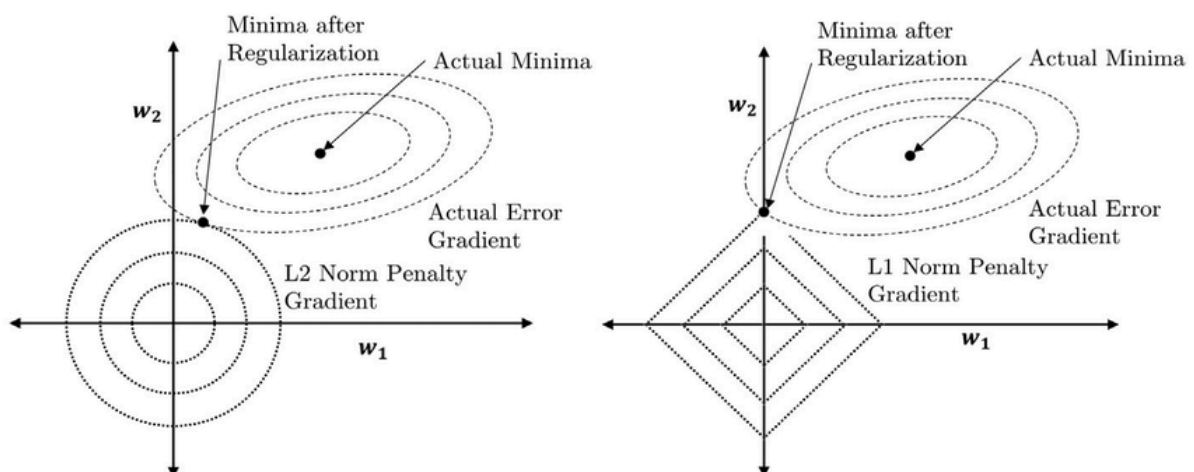
На що ми можемо звернути увагу? В другому випадку бачимо великі вагові коефіцієнти. Давайте додамо обмеження на величину вагових коефіцієнтів в функціонал похибки. Додамо квадратичний регуляризатор в наш функціонал похибки. Тепер він буде мати вигляд

$$Q(w, X) + \lambda \|w\|^2 \rightarrow \min.$$

Змінюючи коефіцієнт λ ми будемо "дозволяти" моделі мати більші значення вагових коефіцієнтів або обмежувати ці значення

Виділяють два типи регуляризації. Квадратичний (L2, Ridge) регуляризатор обмежує квадрати вагових коефіцієнтів. Цей функціонал гладкий і легше диференційований. Графічно можна представити, що ми обмежуємо всі можливі значення вагових коефіцієнтів колом (сферою, гіперсферою) певного радіусу. Ми розуміємо, що для тих даних, на яких модель тренувалась, мінімум функціонала похибки самої моделі може бути поза цим колом і ми, серед всіх можливих значень комбінації вагових коефіцієнтів, знайдемо точку, що лежить на колі. В scikit-learn як параметр L2 регуляризатора задають C - величину, що обернена до λ (змінюється в межах від 10^{-8} до 10^8)

- Чим більше C , тим більш складні залежності може описати модель
- Якщо регуляризація сильна (малі значення C), вагові коефіцієнти будуть невеликими, (навіть можуть занулитись) і модель буде недонавченою
- Якщо регуляризація слабка (великі значення C), вагові коефіцієнти можуть бути великими, модель буде "боятись" помилитись на тестових значеннях, тобо буде перенавченою
- Оскільки C - це гіперпараметр, який впливає на процес навчання моделі, він не може бути отриманий в процесі навчання, а налашовується на крос-валідації і підборі гіперпараметрів



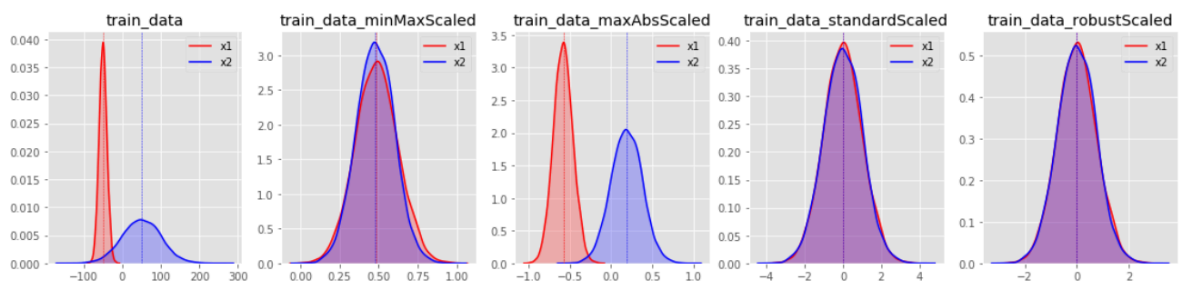
L1 (Lasso) регуляризатор обмежує на квадрати, а абсолютні значення вагових коефіцієнтів. Тобто функціонал помилки має вигляд $Q(w, X) + \lambda \|w\| \rightarrow \min$. На відміну від L2 регуляризатора, L1 регуляризатор не гладка функція, що складніше для тренування моделі. Зона, в якій можуть знаходитись вагові коефіцієнти, обмежена квадратом (кубом, гіперкубом). Через специфіку фігури досить часто оптимальні вагові коефіцієнти після регуляризатора знаходяться не на ребрах (гранях) квадрата, а в вершинах. Це означає, що L1 регуляризатор частіше занулює значення вагових коефіцієнтів. Тому іноді його використовують для відбору значущих незалежних змінних.

Для будь-якого регуляризатора і, відповідно, для лінійних моделей важливо, щоб значення в кожній колонці датасету були приблизно в однакових межах. Цей процес називають нормалізація.

Мета **нормалізації** – змінити значення числових стовпців у наборі даних для використання загальної шкали без спотворення відмінностей у діапазонах значень чи втрати інформації.

В scikit-learn реалізовано кілька методів масштабування даних.

MinMaxScaler - $X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$. Вираховує різницю між максимальним і мінімальним значенням кожної колонки. Мінімальне значення стає 0, максимальне - 1, всі проміжні значення розподіляються пропорційно в діапазоні від 0 до 1



MaxAbsScaler - $X' = \frac{X}{X_{\max}}$. Ділить кожне значення в колонці на максимальне в цій колонці. Не дуже добре працює, коли максимальне позитивне значення за модулем сильно відрізняється від максимального негативного значення. Нові значення на позитивному напрямі

розподіляються пропорційно в діапазоні від 0 до 1, на негативному напрямі - залежить від значень

StandardScaler. Визначає математичне очікування значень в кожній колонці (середнє значення) і стандартне відхилення.

Standardization:

$$z = \frac{x - \mu}{\sigma}$$

with mean:

$$\mu = \frac{1}{N} \sum_{i=1}^N (x_i)$$

and standard deviation:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

RobustScaler масштабує значення відкинувши математичне очікування в колонці таким чином, щоб значення, які знаходяться між першим і третім квантилем були в діапазоні від -1 до 1

Масштабування даних є критичним для лінійних, метричних та багатьох інших моделей. Підбір типу скалера також можна віднести до гіперпараметра, але не моделі, а процесу підготовки даних

▼ Крос-валідація

Зазвичай датасет розбивають на 2-3 частини. На одній тренують модель, а на інших перевіряють. Іноді дані в датасеті бувають впорядковані в певному порядку, наприклад відсортовані по таргету, або по одній із колонок із даними. Якщо ми розіб'ємо датасет на дві частини (зазвичай співвідношення 80/20, 70/30 або 67/33), то є ймовірність того, що дані в тренувальній частині можуть відрізнатись від даних в тестовій частині.

Щоб впевнитись в тому, що дані в датасеті розподілені рівномірно роблять кросс-валідацію. Кросс-валідація - це техніка, при якій датасет розбивають на кілька частин і послідовно одну з частин залишають для перевірки, а інші використовують для тренування моделі. Схематично це показано на малюнку



Метрики, отримані при крос-валідації є середніми значеннями метрик по ітераціям. В scikit-learn є різні методи реалізації тренування з крос-валідацією. Наприклад, є клас RidgeCV, який дозволяє підібрати коефіцієнт регуляризації

```
from sklearn import datasets, linear_model
from sklearn.linear_model import RidgeCV
X, y = datasets.load_diabetes(return_X_y=True)
clf = RidgeCV(alphas=[1e-3, 1e-2, 1e-1, 1], cv=4).fit(X,
y)
clf.score(X, y), clf.intercept_, clf.coef_
>>> (0.5177068677331569,
152.133484162896,
array([ -9.54916175, -239.08695779, 520.3693746 , 323.
82274522,
-712.32215918, 413.37912498, 65.81132269, 167.
51300694,
720.9399241 , 68.12336029]))
```

В більш загальному варіанті є клас GridSearchCV, який приймає модель та словник її параметрів для перебору

```

from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import SGDRegressor
parameters = {'penalty':('l2', 'l1', 'elasticnet'), 'l1_ratio':[0, 0.5, 1], 'alpha':[ 1e-2, 1, 100]}
sgd_r = SGDRegressor(max_iter=10000)
clf = GridSearchCV(sgd_r, parameters)
clf.fit(X, y)
clf.best_params_
>>>{'alpha': 0.01, 'l1_ratio': 0.5, 'penalty': 'l1'}

```

Є випадки, коли крос-валідація відбувається по-іншому. Наприклад, при аналізі часових рядів не можна тренувати модель на даних, які відбулись пізніше, ніж тестові. В цьому випадку крос-валідацію роблять поступово розширюючи тренувальну частину, а тестову обирають одразу за тренувальною

